

Constraining Mo Elastic Constants in MACE-MPA-0

Contents

Constraining Mo Elastic Constants in MACE-MPA-0	1
What MACE-MPA-0 computes	1
Why this can be a quadratic program	2
Getting D out, and writing $\delta\Theta$ back in	2
Constructing the elastic constraints	3
The quadratic program	4
Why the linearisation can be trusted	5
Results	5
Open choices and caveats	6
Running it	7

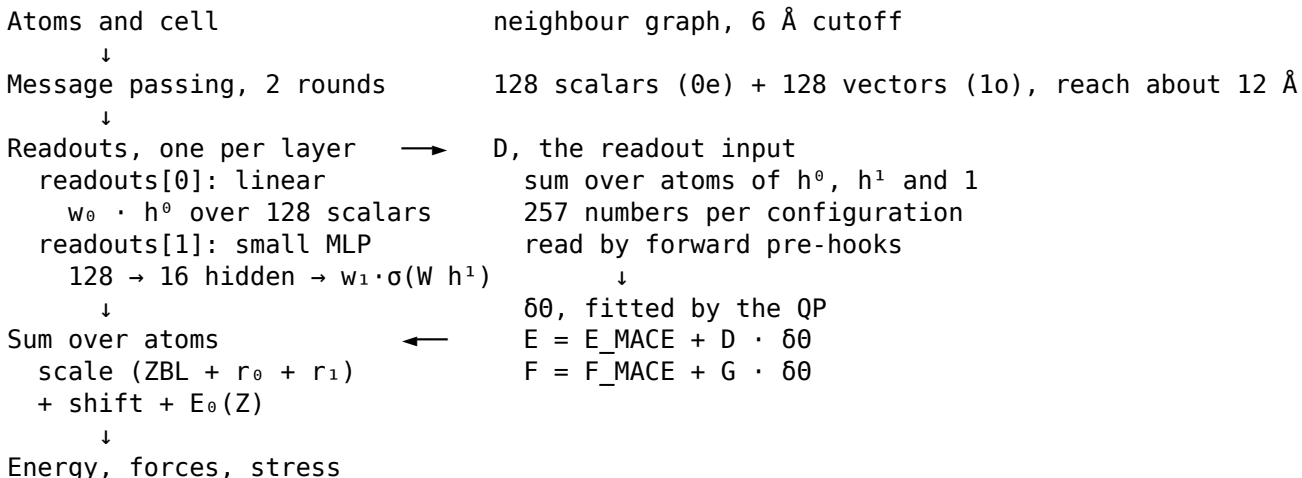
Constraining Mo Elastic Constants in MACE-MPA-0

Written 2026-10-02. Living version, with the architecture diagram and the force-weight chart:
<https://claude.ai/code/artifact/e98e7b41-210e-4441-ab2f-e83e57cf0703>

A readout-only linear corrector that puts MACE-MPA-0's Mo elastic constants on experiment, following Constrain_Mb_Elastics.pdf. Code: constrained_optimization/mo_mace/.

What MACE-MPA-0 computes

MACE turns a configuration into an energy in four stages, and only the last one carries parameters we touch:



Every atom within 6 Å of another is an edge of a graph. Two rounds of message passing then give each atom a feature vector built from its neighbours' relative positions, so the effective reach is

about 12 Å. Those features are *equivariant*, labelled 128x0e+128x1o: 128 scalars that are rotation-invariant, and 128 vectors that rotate with the crystal. Energy is a scalar, so only the scalars can feed it.

The assembled energy, from `ScaleShiftMACE.forward`:

$$E = \sum_{\text{atoms}} E_0(Z) + \sum_{\text{atoms}} \left[\text{scale} \cdot (\text{ZBL}_{\text{pair}} + w_0 \cdot h^0 + w_1 \cdot \sigma(Wh^1)) + \text{shift} \right]$$

For this checkpoint `scale = 0.7736` and `shift = 0`. $E_0(Z)$ is a fixed per-element reference.

The ZBL term is a trap worth knowing about. It is a short-range nuclear repulsion whose cutoff is the sum of covalent radii, 3.08 Å for Mo, while the BCC nearest-neighbour distance is 2.73 Å. So it is active at equilibrium and it contributes to the elastic constants. It carries no free parameters here, so it belongs in the frozen baseline — but the first version of the consistency check omitted it and would have failed the job.

Why this can be a quadratic program

MACE’s energy is linear in the readout coefficients, once everything upstream of them is frozen. Quadratic programming needs exactly that.

Look at the last line of each readout: the energy depends on w_0 and w_1 linearly, with the features feeding them just numbers. So the message passing is frozen and only a correction to the readout is fitted. Per configuration, define

$$D = \sum_{\text{atoms}} [h^0 \text{ (128 scalars)}, h^1 \text{ (128 scalars)}, 1] \in \mathbb{R}^{257}$$

and the corrected model is

$$E(\delta\Theta) = E_{\text{MACE}} + D \cdot \delta\Theta$$

$\delta\Theta = 0$ is the untouched foundation model, so the fit measures a correction rather than relearning the potential. This is Perez et al. Eq. 10 and the spec’s Eq. 2: D is the scalar *input* to the readout layer, 256 numbers for MACE-MPA-0. The trailing 1 counts atoms and absorbs the constant per-atom offset between the mlearn PBE reference and the MPtrj reference MACE was trained on.

`DESCRIPTOR=readout` is the alternative: take the 16 MLP hidden units instead of the 128 layer-1 scalars. Then $\delta\Theta$ is literally a change to the existing readout weights, but it has 16 layer-1 directions instead of 128, so it is much less expressive. Both are exactly linear.

Getting D out, and writing $\delta\Theta$ back in

MACE exposes neither the readout inputs nor a way to add a linear head, so `lib_mace_readout.py` does both by hand.

Reading D . `ReadoutHooks` registers PyTorch forward pre-hooks on `readouts[0]`, `readouts[1]` and `readouts[1].linear_2`, which capture each block’s input tensor as it is called. `energy_and_descriptor` runs an ordinary energy evaluation and sums the captured features over atoms. One forward pass gives E_{MACE} and D together.

Writing $\delta\Theta$ back. `apply_correction` returns a real MACE model, not just a coefficient vector:

- The layer-0 part is folded into `readouts[0].linear`’s weights. `e3nn` normalises its weights internally, so the code **probes** the effective weight by passing unit vectors through the layer (`_probe`) instead of assuming a normalisation convention.

- The layer-1 part cannot be folded into the MLP, because $\delta\Theta$ multiplies the MLP's *input* rather than its output. `readouts[1]` is therefore wrapped in `LinearCorrectedReadout`, which adds $c \cdot h^1$ alongside the original block.
- The offset goes into `scale_shift.shift`.
- Everything is divided by `scale` (0.7736), because the readout sum is scaled afterwards.

The result is saved as `mace_mpa0_mo_<fit>.model` and loads through `MACECalculator`, so it can drive MD or phonons like any other MACE model. Because the perez pickle references `LinearCorrectedReadout`, the `constrained_optimization/mo_mace` directory must be on `PYTHONPATH` when loading it.

Forces come from the same construction: $F(\delta\Theta) = F_{\text{MACE}} + G \cdot \delta\Theta$ with $G = -\partial D / \partial r$, computed by reverse-mode autograd in blocks of 8 descriptor columns. Blocking matters only for memory: all 256 columns at once needs about 40 GB per worker on a 54-atom cell, 8 at a time needs 3.8 GB, and the two agree to $4e-16$.

Constructing the elastic constraints

The whole elastic requirement is a 4×257 matrix with lower and upper bounds, built in `constraints()` in `01_build_design_and_constraints.py`. The step that makes it work is differentiating D , not E .

Elastic constants are second derivatives of energy with respect to strain:

$$C_{ij} = \frac{1}{V} \frac{\partial^2 E}{\partial \varepsilon_i \partial \varepsilon_j}$$

1. Pick the reference crystal. `relaxed_bcc_a` minimises $E(a)$ for MACE-MPA-0 by two quadratic refinements, giving $A0 = 3.1512 \text{ \AA}$. It builds the 2-atom cubic BCC cell and records its volume $V = 31.29 \text{ \AA}^3$.

2. Strain the cell. `strained()` applies $\text{cell}' = \text{cell} \cdot (I + \varepsilon)^T$ with the atoms scaled along, in Voigt convention, so $\varepsilon_4 = 2\varepsilon_{yz}$. `strain_derivatives` evaluates central differences with step $h = 0.005$:

Derivative	Stencil	Gives
d11	$[f(+h) - 2f(0) + f(-h)] / h^2$, straining ε_1	C11
d12	$[f(++) - f(+-) - f(-+) + f(--)] / 4h^2$, straining ε_1 and ε_2	C12
d44	$[f(+h) - 2f(0) + f(-h)] / h^2$, straining ε_4	C44
d1	$[f(+h) - f(-h)] / 2h$, straining ε_1	stress

BCC is a Bravais lattice, so every atom sits on an inversion centre and uniform strain needs no internal relaxation. That is why a 2-atom cell suffices and why no geometry optimisation appears anywhere in the constraint build.

3. Differentiate the descriptor. The same stencil is applied twice: once to the energy, giving MACE's own C_{ij} , and once to D , giving the second derivative of each of the 257 descriptor columns. So `dD["d11"]` is a 257-vector. Because the corrected energy is $E_{\text{MACE}} + D \cdot \delta\Theta$ and differentiation is linear,

$$C_{11}(\delta\Theta) = C_{11}^{\text{MACE}} + \frac{1}{V} \frac{\partial^2 D}{\partial \varepsilon_1^2} \cdot \delta\Theta$$

The elastic constant of the corrected model is an **exactly linear function of $\delta\Theta$** . That is what makes this a linear constraint rather than a nonlinear optimisation, with no approximation beyond the finite-difference step.

4. Assemble rows and bounds. With $k = 160.21766/V$ converting $\text{eV}/\text{\AA}^3$ to GPa, each row is $k * \text{dD}[\text{key}]$ and the bounds subtract what MACE already delivers, because OSQP constrains $A \cdot \delta\Theta$, the *change*:

$$l_c = C_c^{\text{exp}}(1 - \text{tol}) - C_c^{\text{MACE}}, \quad u_c = C_c^{\text{exp}}(1 + \text{tol}) - C_c^{\text{MACE}}$$

For C11: the target is $463.7 \pm 1\%$ against MACE's 518.4, so the correction must deliver between -59.3 and -50.0 GPa.

5. The fourth row pins the minimum. It is $k * \text{dD}["d1"]$ with bounds $\pm P_TOL - \sigma_MACE$. By cubic symmetry $\partial E / \partial \varepsilon_1 = \partial E / \partial \varepsilon_2 = \partial E / \partial \varepsilon_3$ and the shears vanish, so this single row means zero pressure. It is needed for two reasons:

- $C_{ij} = V^{-1} \partial^2 E / \partial \varepsilon_i \partial \varepsilon_j$ equals the physical stress-strain elastic constants only at zero stress.
- Without it the correction drifts the lattice constant. Unconstrained ridge moved it by $0.018 \text{ \AA}$; the constrained fit moved it by $0.0004 \text{ \AA}$.

The rows and bounds are written to `A_con.csv`, `l_con.csv` and `u_con.csv`, with a readable summary in `con_meta.txt`.

The quadratic program

`02_constrained_readout_qp.jl` assembles the spec's Eq. 1 and hands it to OSQP.jl, which solves 257 variables against 4 constraint rows in under 0.1 s.

$$\min_{\delta\Theta} \quad \frac{1}{2} \delta\Theta^\top P \delta\Theta + q^\top \delta\Theta \quad \text{s.t.} \quad l \leq A \delta\Theta \leq u$$

The objective, in MACE's own loss weighting:

$$\frac{1}{2} \left(\frac{E_{\text{DFT}} - E_{\text{MACE}} - D \cdot \delta\Theta}{N} \right)^2 + \frac{1}{2} w_F (F_{\text{DFT}} - F_{\text{MACE}} - G \cdot \delta\Theta)^2 + \frac{1}{2} \lambda \|S \delta\Theta\|^2$$

- **Energy term.** Rows are per atom: D and the residual are both divided by N , so a 53-atom cell does not outweigh a 2-atom one.
- **Force term.** G is never stored. Step 01 accumulates $G^\top G$ (257×257), $G^\top \Delta F$ and $\Delta F^\top \Delta F$ per split and per config group, which is everything the normal equations need. It also makes force RMSE exact for any $\delta\Theta$ from $\|\Delta F - G\delta\Theta\|^2 = \text{ftf} - 2\delta\Theta^\top G^\top \text{ftf} + \delta\Theta^\top G^\top G \delta\Theta$, so the weight scan costs nothing. 30,261 train and 3,567 test force components.
- **Ridge λ .** Pulls $\delta\Theta$ towards 0, which means towards the foundation model. The offset column is exempt (`pen[end] = 0`), since it is a reference shift rather than physics.
- **Column scaling S .** Raw descriptor columns differ by orders of magnitude, so the problem is solved in $\theta = S \cdot \delta\Theta$ with S the column standard deviations. The constraint matrix is scaled identically ($A_s = A * S^{-1}$) and the solution is unscaled on return, so the constraints mean the same thing.
- **Tolerances.** `eps_abs = eps_rel = 1e-9` with polish, not OSQP's `1e-3` defaults. That is the lesson from the W positive-core QP, where loose tolerances on a badly scaled problem left the solution undetermined.

Three fits are reported side by side so the constraint’s cost is visible: **offset** (the constant column alone, i.e. MACE-MPA-0 with its reference shifted), **ridge** (all columns, no constraints) and **constrained**.

Why the linearisation can be trusted

Everything above is valid only if D really is MACE’s internals. Step 01 therefore takes a **random** $\delta\Theta$, writes it into a real MACE model, and checks the model’s own output against the linear prediction before any expensive work begins. Measured in job 6202749:

Check	Residual	Scale of the thing being checked
$E_{\text{patched}} - (E_{\text{MACE}} + D \cdot \delta\Theta)$	1e-13 eV	$D \cdot \delta\Theta \approx 184 \text{ eV}$
$F_{\text{patched}} - (F_{\text{MACE}} + G \cdot \delta\Theta)$	1.3e-14 eV/Å	$\max G \cdot \delta\Theta = 1.49 \text{ eV/Å}$
$\partial E_{\text{patched}} / \partial \epsilon - \partial E_{\text{MACE}} / \partial \epsilon$ – $\partial D / \partial \epsilon \cdot \delta\Theta$	below the finite-difference floor	$\partial^2 E / \partial \epsilon_1^2 \approx 90 \text{ eV}$

These run first by design: a mistake in the descriptor fails within minutes instead of after an hour of force Jacobians.

Step 03 then rebuilds the corrected model from the fitted $\delta\Theta$ and recomputes C_{ij} , the relaxed lattice constant, and the energy and force errors **from the patched model itself**, never reusing the linear prediction. That is why C11 at A0 (468.34 GPa) and at the model’s own relaxed lattice (467.75 GPa) differ slightly: they are independent recomputations, not the same number reported twice.

Step 04 goes one step further and loads the saved .model file from disk, so the artefact that would drive an MD run is the one being measured.

Results

The constrained model sits within 1.2% of experiment on all three elastic constants, and improves energies and forces at the same time. Job 6202749, 1 h 31 m, DESCRIPTOR=perez, $w_F = 1$, $\lambda = 1e-4$.

Model	C11	C12	C44	a (Å)	Test E RMSE	Test F RMSE
Experiment (273 K)	463.7	157.8	109.2			
MACE-MPA-0	518.4 (+11.8%)	201.1 (+27.5%)	85.0 (−22.2%)	3.1512	21.9 meV/atom	219 meV/Å
Ridge, unconstrained	638.2 (+37.6%)	231.9 (+46.9%)	106.6 (−2.4%)	3.1694	8.0 meV/atom	171 meV/Å
Constrained	467.8 (+0.9%)	159.0 (+0.8%)	107.9 (−1.2%)	3.1516	15.3 meV/atom	172 meV/Å

C_{ij} in GPa, each model at its own relaxed lattice constant. Elastic constants and lattice constant are recomputed from the patched model by finite differences; errors are on the 23 held-out mlearn test configs.

The constraint earns its place. Unconstrained ridge gives the best energies but drives C11 to 638 GPa, which is *worse than the original model* and 38% above experiment. Fitting Mo energies

and forces alone does not fix the elastic constants; it damages them. Only four rows of the 257-variable problem are elastic, and they cost 7.3 meV/atom of test energy error relative to ridge.

Forces improved rather than degraded. The earlier energy-only fit pushed test force error from 219 up to 330 meV/Å. Adding forces to the objective brings it to 172 meV/Å, 21% better than MACE-MPA-0, while test energy error still falls from 21.9 to 15.3 meV/atom.

The force-weight scan from step 02 (test set; see the doc for the plotted trade-off):

w_F	Test E RMSE (meV/atom)	Test F RMSE (meV/Å)
0	7.85	330.0
0.01	6.27	180.7
0.1	8.71	174.3
1 (run)	15.34	172.1
10	26.55	172.1
100	35.13	172.2

The force error is already flat by $w_F = 0.01$, while the energy error keeps climbing with weight. The run used $w_F = 1$, but **$w_F = 0.01$ looks the better operating point**. Steps 02–04 re-solve from the stored matrices in seconds, so testing it costs nothing.

The minimum stayed put. $3.1512 \rightarrow 3.1516$ Å, as the zero-stress row requires.

MACE-MPA-0’s own 219 meV/Å force error on this set is high for a foundation model on an elemental metal and is worth a separate look; it is not something the corrector introduced.

Open choices and caveats

Five things the spec left open or that need judgement before these numbers go in a paper.

The experimental C_{ij} could not be sourced from Smirnova. The spec says to take them from Smirnova et al., PRMaterials 4, 013605 (2020), which is paywalled with no arXiv copy. The defaults are Dickinson & Armstrong 1967 at 273 K, 463.7/157.8/109.2 GPa, as tabulated in [Dal Corso, arXiv:2406.16634](#), Table II. **Check these against Smirnova’s table.** Other rows in that table differ materially: Bolef & de Klerk at 300 K give 469.6/167.6/106.8, and the 0 K extrapolation gives 480.0/155.8/112.4. Override with $C11=$ $C12=$ $C44=$.

Temperature mismatch. MACE is a static-lattice 0 K model with no zero-point motion, but the defaults are 273 K measurements. The 273 K $\rightarrow$ 0 K shift is about 3% on $C11$, three times the $\pm 1\%$ window, so the choice of row matters more than the tolerance.

PBE fights the experimental target. PBE itself puts $C44$ about 9% below experiment (Dal Corso). The objective uses PBE energies and forces while the constraint uses experiment, so the two pull against each other by construction. That is visible as the 7.3 meV/atom gap between ridge and constrained.

The constraints hold at one volume only. All four rows are evaluated at $A0$. Nothing is constrained away from that volume, in other structures, or at defects. We have been caught by exactly this on Al: constraining only at the equilibrium lattice constant looked fine until NPT runs heated the lattice and members left FCC. If this Mo model is going into MD at temperature, add the same rows at a second expanded volume, around $1.02\text{--}1.05 \cdot A0$. That is a small change to step 01.

OSQP reported solution polish: unsuccessful on the force run, unlike the energy-only run. The iterates converged tightly (primal and dual residuals $3e-13$) and the constraints are satisfied in the independent step-03 recomputation, so this looks harmless, but it is a difference worth knowing about before quoting $\delta\Theta$ as reproducible.

The DFT set is a choice too. The spec named no dataset; this uses the public mlearn Mo set (Zuo et al., JPCA 124, 731 (2020)): PBE, 194 train and 23 test configs across Elastic, AIMD-NVT, Vacancy and Surface groups. Its PBE reference differs from MPtrj's by a constant per atom, which the offset column absorbs.

Running it

One command, about 1 h 30 m on one compute node:

```
cd /storage/astro2/phupfb/PhD/acestuff/ACEWorkflow
sbatch constrained_optimization/mo_mace/run_pipeline.slurm
```

The five steps, in order:

1. `00_fetch_mlearn_mo.py` — downloads the mlearn Mo set and converts it to extxyz in `data/Mo/`. Cached, so compute nodes need no internet.
2. `01_build_design_and_constraints.py` — exactness checks, then D and the force normal equations for all 217 configs over 4 worker processes, then the constraint rows. This is the expensive step, about 1 h.
3. `02_constrained_readout_qp.jl` — the OSQP solve, seconds. Prints the force-weight scan and the three fits.
4. `03_verify_corrected_model.py` — patches the model, recomputes everything independently, saves the `.model` files.
5. `04_before_after.py` — the before/after table, also written to `before_after.csv`.

Outputs land in `models/Mo_MACE_MPA0_readout/<DESCRIPTOR>/`.

Changing the force weight does not need step 01 again. Steps 02-04 re-solve in seconds from the stored matrices:

```
FORCES_WEIGHT=0.01 julia --project constrained_optimization/mo_mace/02_constrained_readout_qp.jl
FORCES_WEIGHT=0.01 python/mace_venv/bin/python constrained_optimization/mo_mace/03_verify_corrected_model.py
python/mace_venv/bin/python constrained_optimization/mo_mace/04_before_after.py
```

Variable	Default	What it does
DESCRIPTOR	perez	257-column readout input, or readout for the 145-column weight reparameterisation
FORCES_WEIGHT	1.0	force weight w_F ; 0 gives an energy-only fit
C11 C12 C44	463.7 / 157.8 / 109.2	experimental targets in GPa
REL_TOL	0.01	half-width of the C_{ij} windows
P_TOL	0.1	zero-stress tolerance in GPa
A0	mace	constrain at MACE's relaxed a , or exp for 3.147 Å, or a number
LAMBDA	1e-4	ridge towards MACE-MPA-0
STRAIN_H	0.005	finite-difference strain step
N_WORKERS / JAC_BLOCK	4 / 8	parallelism and Jacobian memory/speed trade-off

The environment is `python/mace_venv` (mace-torch 0.3.16, CPU torch 2.8), which is gitignored; the README records how it was built. Code is on branch `worktree-mo-mace-elastics`.